**UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA**

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS · ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

# TECNOLOGÍA DE OBJETOS

## SGTE-UNSA: SISTEMA DE GESTIÓN DE TRÁMITES Y EXPEDIENTES ACADÉMICOS EN C++17 Y QT 6

*PROYECTO FINAL · POO, PATRONES DE DISEÑO GoF, STL Y MULTITHREADING · GRUPO B (2026-B)*

**Docente:**

Mg. Roni Guillermo Apaza Aceituno

**Integrantes del Equipo:**

1. Choquehuanca Bedoya, Brayan Denilson
2. Callo Ccagiavilca, Kevin Joel
3. Ticona Saure, Jose Maria

**ENFOQUE DEL PROYECTO**

Aplicación de escritorio con arquitectura MVC que integra herencia polimórfica, punteros inteligentes (`std::shared_ptr`), plantillas genéricas (`Repositorio<T>`), expresiones lambda con STL, patrones GoF (**Singleton**, **Factory Method**, **Strategy**, **Decorator**) y auditoría concurrente con `QThread`.

# 1. Propósito del Sistema y Arquitectura MVC + Singleton

*ARQUITECTURA DE SOFTWARE Y CONTEXTO INSTITUCIONAL*

## Problemática y Solución (SGTE-UNSA)

- Los trámites universitarios (*Matrícula*, *Carta de Prácticas*, *Grado de Bachiller*) tienen **tasas, prioridades y descuentos distintos**.
- **SGTE-UNSA** desacopla las reglas de negocio de la interfaz gráfica mediante C++17 y Qt 6 Widgets.
- Persistencia inmediata en archivos planos (`tramites_unsa.csv`) sin requerir servidores externos.

## Patrón Arquitectónico MVC + Singleton

- **Modelo (`Tramite`, `Repositorio<T>`):** Entidades polimórficas, cálculo de tasas y almacenamiento genérico.
- **Vista (`MainWindow`):** Formulario, tabla interactiva y barra de progreso en Qt Widgets.
- **Controlador (`ControladorTramites`):** Coordina eventos de la Vista hacia el Modelo.
- **Singleton (`ConfiguracionUNSA`):** Instancia única con el semestre activo (`2026-B`) y ruta del archivo CSV.

## Flujo de Comunicación entre Capas

`Usuario (Qt GUI)` → `MainWindow (Vista)` → `ControladorTramites (Controlador)` → `TramiteFactory / Repositorio<Tramite> (Modelo)` → `tramites_unsa.csv`

# 2. Encapsulación, Herencia y Polimorfismo Dinámico



## Jerarquía Polimórfica (Tramite.h)

- **Clase Base Abstracta `Tramite`:**
  - Atributos protegidos: `codigo_`, `dniEstudiante_`, `nombreEstudiante_`, `tasaBase_`, `estado_`.
  - Métodos virtuales puros (`= 0`):
    - `calcularCostoFinal() const`
    - `obtenerPrioridad() const`
    - `obtenerTipo() const`
- **3 Subclases Concretas (`override`):**
  1. `TramiteMatricula`: Prioridad 2 (Media) | Base `S/ 30.00`.
  2. `TramiteCartaPracticas`: Prioridad 3 (Normal) | Base `S/ 15.00`.
  3. `TramiteGradoBachiller`: Prioridad 1 (Alta) | Base `S/ 180.00 + S/ 20.00` (diploma).

```cpp
class Tramite {
protected:
    std::string codigo_, dniEstudiante_, nombreEstudiante_;
    double tasaBase_;
    std::shared_ptr<IEstrategiaTasa> estrategiaTasa_;
public:
    virtual ~Tramite() = default;
    virtual std::string obtenerTipo() const = 0;
    virtual double calcularCostoFinal() const = 0;
    virtual int obtenerPrioridad() const = 0;
};

class TramiteGradoBachiller : public Tramite {
public:
    std::string obtenerTipo() const override {
        return "Grado Bachiller";
    }
    double calcularCostoFinal() const override {
        return estrategiaTasa_->calcularMonto(tasaBase_) + 20.0;
    }
    int obtenerPrioridad() const override { return 1; }
};
```

# 3. Patrón Factory Method y Punteros Inteligentes

*PATRONES CREACIONALES Y GESTIÓN AUTOMÁTICA DE MEMORIA*

## Desacoplamiento con TramiteFactory

- La ventana Qt (`MainWindow`) **nunca invoca `new`** ni conoce las subclases concretas.
- Envía los textos del formulario a `TramiteFactory::crearTramite(...)`.
- Uso de **`std::shared_ptr<Tramite>`** y **`std::make_shared`** (`<memory>`): liberación automática RAII sin fugas de memoria (*memory leaks*).

```
std::shared_ptr<Tramite> TramiteFactory::crearTramite(
    const std::string& tipo, const std::string& codigo,
    const std::string& dni, const std::string& nombre,
    const std::string& modalidadTasa, bool urgente)
{
    auto estrategia = crearEstrategia(modalidadTasa);
    std::shared_ptr<Tramite> nuevo;

    if (tipo == "Matricula")
        nuevo = std::make_shared<TramiteMatricula>(...);
    else if (tipo == "Carta Practicas")
        nuevo = std::make_shared<TramiteCartaPracticas>(...);
    else if (tipo == "Grado Bachiller")
        nuevo = std::make_shared<TramiteGradoBachiller>(...);

    if (urgente)
        nuevo = std::make_shared<TramiteUrgenteDecorator>(nuevo);
    return nuevo;
}
```

# 4. Patrones GoF: Strategy (Tasas) y Decorator (Urgencia)

*PATRONES ESTRUCTURALES Y DE COMPORTAMIENTO*

## Patrón Strategy (EstrategiaTasa.h)

- Interfaz `IEstrategiaTasa` con método virtual puro `calcularMonto(double tasaBase) const`.
- **3 Estrategias intercambiables:**
  1. `EstrategiaTasaRegular`: cobra el **100%** de la tasa.
  2. `EstrategiaTasaBeca`: aplica **50% de descuento**.
  3. `EstrategiaTasaExonerado`: costo **S/ 0.00**.
- Evita bloques condicionales `if/else` dentro de cada trámite.

## Patrón Decorator (TramiteUrgenteDecorator)

- Envuelve dinámicamente cualquier `std::shared_ptr<Tramite>` cuando se marca *«Atención Urgente»*.
- **Sin modificar las subclases (Principio OCP - SOLID):**
  - Añade `[URGENTE]` al nombre del tipo.
  - Suma `+S/ 20.00` al resultado de `calcularCostoFinal()`.
  - Eleva la prioridad a **`0 - URGENTE`**.

```
class TramiteUrgenteDecorator : public Tramite {
    std::shared_ptr<Tramite> tramiteEnvuelto_;
public:
    double calcularCostoFinal() const override { return tramiteEnvuelto_->calcularCostoFinal() + 20.0; }
    int obtenerPrioridad() const override { return 0; }
};
```

# 5. Sobrecarga de Operadores, inline/const/friend y Excepciones

*CARACTERÍSTICAS AVANZADAS DE C++ Y ROBUSTEZ*

## C++ Avanzado en Tramite

- **`inline` y `const`:** Accesores seguros (`getCodigo() const`, `getDni() const`) sin costo de llamada.
- **Sobrecarga de `operator<`:** Compara primero `obtenerPrioridad()` (`0 < 1 < 2 < 3`) y luego el código.
- **Sobrecarga de `operator==`:** Detecta expedientes duplicados por código.
- **Función `friend operator<<`:** Formatea el objeto hacia `std::ostream` para el reporte de auditoría.

## Jerarquía de Excepciones (Excepciones.h)

- Hereda de `std::runtime_error`:
  - `ExcepcionTramite` (clase base)
  - `ExcepcionValidacion` (DNI inválido $\neq 8$ dígitos, campos vacíos o código duplicado)
  - `ExcepcionPersistencia` (fallo de apertura CSV)
- Capturadas con `try / catch` en la interfaz Qt mostrando diálogos `QMessageBox::warning`.

```
bool operator<(const Tramite& otro) const {
    if (this->obtenerPrioridad() != otro.obtenerPrioridad())
        return this->obtenerPrioridad() < otro.obtenerPrioridad();
    return this->codigo_ < otro.codigo_;
}
```

# 6. Plantilla Repositorio<T> y Almacenamiento en CSV



## Diseño Genérico (Repositorio.h)

- Clase plantilla **`template <typename T> class Repositorio`** independiente del tipo de entidad.
- Respaldada internamente por `std::vector<std::shared_ptr<T>>` garantizando memoria contigua para los punteros y desacoplamiento de la GUI.
- **Persistencia automática:** Cada alta, baja o cambio de estado invoca `guardarEnCSV("tramites_unsa.csv")`.

```
template <typename T>
class Repositorio {
private:
    std::vector<std::shared_ptr<T>> elementos_;
public:
    void agregar(std::shared_ptr<T> item) {
        auto it = std::find_if(elementos_.begin(), elementos_.end(),
            [&item](const auto& act) { return *act == *item; });
        if (it != elementos_.end())
            throw ExcepcionValidacion("Codigo duplicado");
        elementos_.push_back(std::move(item));
    }

    template <typename Predicado>
    std::vector<std::shared_ptr<T>> filtrar(Predicado pred) const {
        std::vector<std::shared_ptr<T>> res;
        std::copy_if(elementos_.begin(), elementos_.end(),
                     std::back_inserter(res), pred);
        return res;
    }
};
```

# 7. Expresiones Lambda, Algoritmos STL y Señales en Qt

*PARADIGMA FUNCIONAL Y LIBRERÍA ESTÁNDAR STL*

## Algoritmos STL con Lambdas en Acción

1. **`std::find_if`:** Busca expedientes por código y previene duplicados.
2. **`std::copy_if`:** Filtra la tabla en tiempo real según texto y estado.
3. **`std::count_if`:** Calcula cuántos trámites están *Pendientes* y cuántos son *Urgentes*.
4. **`std::accumulate`:** Suma la liquidación total en soles (`S/ 372.50`).
5. **`std::sort`:** Ordena toda la colección por prioridad invocando `operator<`.

```
double calcularMontoTotal() const {
    return std::accumulate(elementos_.begin(), elementos_.end(), 0.0,
        [](double acum, const std::shared_ptr<T>& item) {
            return acum + item->calcularCostoFinal();
        });
}

void ordenarPorPrioridad() {
    std::sort(elementos_.begin(), elementos_.end(),
        [](const auto& a, const auto& b) { return *a < *b; });
}

connect(txtBuscar_, &QLineEdit::textChanged, this,
        [this](const QString&) { actualizarTabla(); });
```

# 8. Multithreading con QThread y Objeto Función (Functor)



## Auditoría en Segundo Plano (HiloProcesador.h)

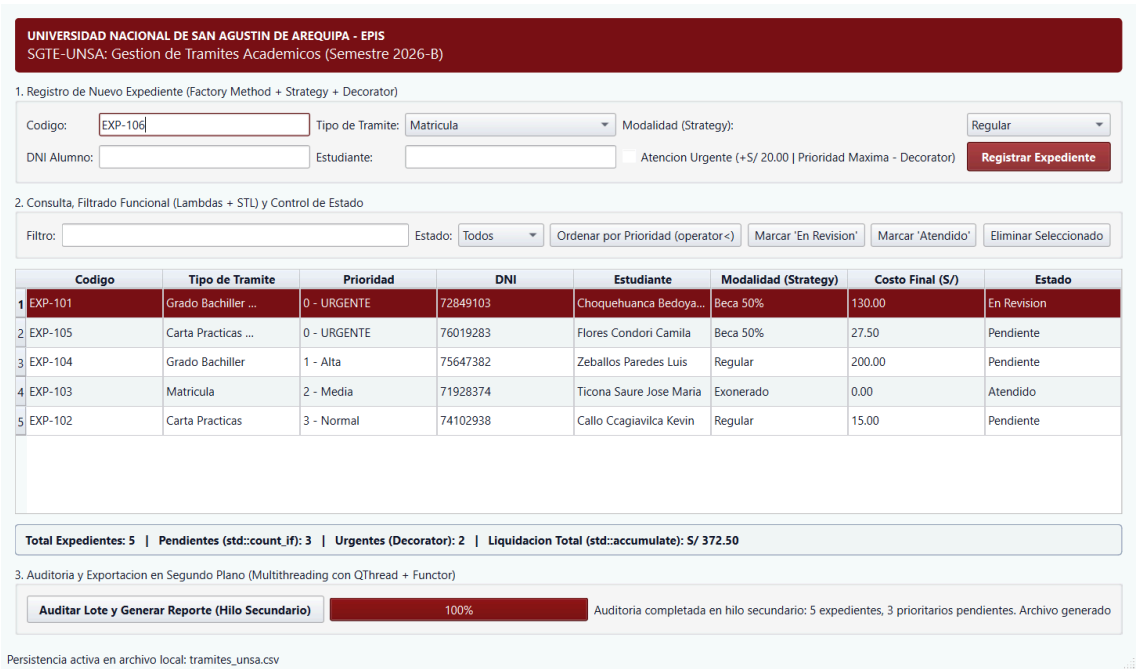
- **Objeto Función (`EvaluadorPrioridadFunctor`):** Sobrecarga `operator()` para detectar expedientes de prioridad alta/ urgente aún no atendidos.
- **`HiloProcesador : public QThread`:**
  - Recibe una copia segura del vector de punteros inteligentes (evita condiciones de carrera).
  - Emite señales `progresoActualizado(int, QString)` hacia el `QProgressBar` de la GUI sin congelar la ventana.
  - Exporta el resumen en `reporte_auditoria.txt`.

```
struct EvaluadorPrioridadFunctor {
    bool operator()(const std::shared_ptr<Tramite>& t) const {
        return t->obtenerPrioridad() <= 1 &&
               t->getEstado() != "Atendido";
    }
};

void HiloProcesador::run() {
    EvaluadorPrioridadFunctor esPrioritario;
    for (int i = 0; i < total; ++i) {
        if (esPrioritario(copiaTramites_[i]))
            ++prioritariosPendientes;
        reporte << (i + 1) << ". " << *copiaTramites_[i] << "\n";
        QThread::msleep(180);
        emit progresoActualizado(((i + 1) * 100) / total, msg);
    }
    emit procesamientoFinalizado(resumen);
}
```

# 9. Demostración de la Interfaz Qt 6 y Conclusiones

*VERIFICACIÓN EXPERIMENTAL Y RESULTADOS*




## Conclusiones del Proyecto

1. **Cobertura Integral:** Integra POO, polimorfismo, `std::shared_ptr`, excepciones, plantillas, STL/lambdas, multithreading y patrones GoF.
2. **Bajo Acoplamiento (SOLID + MVC):** *Strategy* y *Decorator* permiten combinar tasas y urgencias sin multiplicar subclases.
3. **Interfaz Reactiva:** La auditoría en `QThread` mantiene la ventana fluida al 100% mientras genera reportes en disco.